

Assignment 2: Minitorch Framework

The goal of this assignment is to implement a basic deep learning framework, miniTorch, which is capable of performing operations on tensors with automatic differentiation and necessary operators. In this assignment, we will construct a simple feedforward neural network for a sentiment classification task. We will implement the automatic differentiation framework, simple neural network architecture, and training and evaluation algorithms in Python.

Environment Setup

The starting code base is provided in [llmsystem/llmsys_hw2.git](https://github.com/llmsystem/llmsys_hw2).

Please check your version of Python (Python 3.12+), run either:

```
python --version
python3 --version
```

We also highly recommend setting up a virtual environment. The virtual environment lets you install packages that are only used for your assignments and do not impact the rest of the system. We recommend `uv` as it is lightweight and easy to use ([Tutorial](#)) and it makes it possible to select a specific python version for your environment.

Run the following command:

```
uv venv --python=3.12
source .venv/bin/activate
```

Then clone the starter codes from the git repo and install packages.

```
git clone https://github.com/llmsystem/llmsys_hw2.git
cd llmsys_hw2
# If you are using PSC,
# please load the CUDA module before installing packages:
# module load cuda/12.4.0
uv pip install -r requirements.txt
uv pip install -Ue .
```

Make sure that everything is installed by running the following command:

```
python -c "import minitorch; print('Success: minitorch is installed correctly');"
2>/dev/null || echo "Error: Failed to import minitorch. Please check your
installation."
```

Code files layout

```
minitorch/          # The minitorch source code
  autodiff.py       # Automatic differentiation implementation
                    (problem 1)
project/
  run_sentiment.py  # Network and training codes for training for
                    the sentence sentiment classification task
                    (problem 2 & problem 3)
```

Problem 1: Automatic Differentiation (40 points)

Implement automatic differentiation. We have provided the derivative operations for internal Python operators in `minitorch.Function.backward` call. Your task is to write the two core functions needed for automatic differentiation: `topological_sort` and `backpropagate`. This will allow us to traverse the computation graph and compute the gradients along the way.

Complete the following functions in `minitorch/autodiff.py`. The places where you need to fill in your code are highlighted with `BEGIN HW2_1` and `END HW2_1`

Note: Be sure to checkout the functions in `class Variable(Protocol)!`

1. Implement topological sort

Implement the computation for the reversed topological order of the computation graph.

Hints:

- Ensure that you visit the computation graph in a post-order depth-first search.
- When the children nodes of the current node are visited, add the current node at the front of the result order list.

```
def topological_sort(variable: Variable) -> Iterable[Variable]:
    """
    Computes the topological order of the computation graph.
    """
    ...
```

2. Implement backpropagate

Implement the backpropagation on the computation graph in order to compute derivatives for the leave nodes.

```
def backpropagate(variable: Variable, deriv: Any) -> None:
    """
    Runs backpropagation on the computation graph in order to
    compute derivatives for the leave nodes.
    """
    ...
```

3. Check implementation

After correctly implementing the functions, you should be able to pass tests marked as autodiff.

```
python -m pytest -l -v -k "autodiff"
```

Problem 2: Neural Network Architecture (30 points)

In this section, you will implement the neural network architecture. Complete the following functions in `run_sentiment.py` under the project folder. The places where you need to fill in your code are highlighted with `BEGIN HW2_2` and `END HW2_2`.

1. Implement Linear layer

Implement the linear layer with 2D matrix as weights and 1D vector as bias. You need to implement both the initialization function and the forward function for the Linear class. Read the comments carefully before coding.

```
class Linear(minitorch.Module):
    def __init__(self, in_size, out_size):
        ...
    def forward(self, x):
        ...
```

2. Implement Network class

Implement the complete neural network used for training. You need to implement both the initialization function and the forward function of the Network class. Read the comments carefully before coding.

```

class Network(minitorch.Module):
    """
    Implement a MLP for SST-2 sentence sentiment classification.

    This model should implement the following procedure:

    1. Average over the sentence length.
    2. Apply a Linear layer to hidden_dim followed by a ReLU and Dropout.
    3. Apply a Linear to size C (number of classes).
    4. Apply a sigmoid.
    """
    def __init__(
        self,
        embedding_dim=50,
        hidden_dim=32,
        dropout_prob=0.5,
    ):
        ...

    def forward(self, embeddings):
        """
        embeddings tensor: [batch x sentence length x embedding dim]
        """
        ...

```

3. Check implementation

After correctly implementing the functions, you should be able to pass tests marked as network.

```

python -m pytest -l -v -k "linear"
python -m pytest -l -v -k "network"

```

Problem 3: Training and Evaluation (30 points)

In this section, you will implement codes for training and perform training on a simple MLP for the sentence sentiment classification task. The places where you need to fill in your code are highlighted with `BEGIN HW2_3` and `END HW2_3`.

1. Implement cross entropy loss function

You need to implement the binary cross entropy loss function for sentiment classification. This function computes the loss between the predicted output and the true labels. Read the comments carefully before coding.

```
def binary_cross_entropy_loss(out, y):  
    ...
```

2. Implement training loop

You need to complete the code for training and validation. Read the comments carefully before coding. What's more, we strongly suggest leveraging the `default_log_fn` function to print the validation accuracy. The outputs will be used for autograding.

```
class SentenceSentimentTrain:  
    ...  
    The trainer class of sentence sentiment classification  
    ...  
    ...  
    def train(self, data_train, ...):  
        ...
```

3. Training the network

Train the neural network on SST-2 (Stanford Sentiment Treebank) and report your training and validation results.

```
bash run_sentiment.sh
```

You should be able to achieve a best validation accuracy equal to or higher than 75%. It might take some time to download the GloVe embedding file before the first training. Be patient!

Submission

Please submit the whole directory `llmsys_hw2` as a zip on canvas. Your code will be graded with private test cases.

FAQs

1. I cannot get 75% accuracy, what should I do?

We provided the hyperparameters in `run_sentiment.py` for you, but feel free to explore other settings as well (e.g. using SGD/updating learning rate). If you still cannot get more than 75%, please come to the office hour and we can debug together.

1. My automatic differentiation implementation seems correct but tests are failing, what should I do?

Make sure you understand the Variable protocol and the computation graph structure. Pay attention to the order of operations in topological sort and ensure you're handling leaf nodes correctly in backpropagation.

1. Training is taking too long, is this normal?

Training on CPU can take some time, especially for the first epoch. If it's taking unusually long (>30 minutes per epoch), check your implementation for potential inefficiencies or come to office hours.